

Phân đoạn từng đối tượng bằng Deep Learning với StarDist

Đôi chiếu Fiji/ImageJ và cài đặt Python

Nhóm 03 – Học viện Công nghệ Bưu chính Viễn thông

Thành viên: _____

Hạn nộp: 19/11

Tóm tắt

Phân đoạn từng đối tượng (instance segmentation) là bước nền tảng để đếm và đo hình thái tế bào. Báo cáo nghiên cứu StarDist 2D, một phương pháp dùng mạng CNN để dự đoán xác suất tâm đối tượng và khoảng cách theo các tia, sau đó giải mã đa giác star-convex và loại proposal trùng bằng Non-Maximum Suppression (NMS). Nhóm xây dựng pipeline Python tái lập được, tự cài đặt các thao tác hình học và đánh giá instance, đồng thời dùng plugin StarDist của Fiji/ImageJ làm nền tảng tham chiếu. Otsu và marker-controlled Watershed là hai baseline. Kết quả cuối được đánh giá bằng matching một-một theo IoU, AP tại nhiều ngưỡng, Panoptic Quality, Dice foreground, sai số đếm và thời gian xử lý. Các label TIFF Fiji/ImageJ, log tham số và metric của ba ca kiểm chứng được lưu cùng mã nguồn.

Từ khóa

StarDist, instance segmentation, U-Net, star-convex, Fiji, ImageJ, Otsu, Watershed, Panoptic Quality.

Mục lục

1	Giới thiệu	1
1.1	Bối cảnh và động cơ	1
1.2	Mục tiêu và đóng góp	1
1.3	Câu hỏi nghiên cứu	1
2	Cơ sở lý thuyết	2
2.1	Bài toán instance segmentation	2
2.2	CNN và U-Net	2
2.3	Probability map và star-convex polygon	2
2.4	Huấn luyện và NMS	3
2.5	Baseline Otsu và Watershed	3
3	Fiji/ImageJ làm nền tảng tham chiếu	4
3.1	Plugin StarDist	4
3.2	Protocol tái lập và lần chạy thực tế	4
3.3	Kiểm chứng Fiji–Python	4
4	Cài đặt Python	5
4.1	Kiến trúc phần mềm	5
4.2	Phân nhóm tự cài đặt và phân tái sử dụng	5
4.3	Tái lập môi trường	5
5	Thiết kế thí nghiệm và metric	6
5.1	Dữ liệu và split	6
5.2	Matching và metric	6
5.3	Protocol benchmark	6
6	Kết quả và thảo luận	7
6.1	Benchmark tái lập được trên DSB2018	7
6.2	Cách đọc kết quả	10
6.3	Đe dọa tới tính hợp lệ	10

7	Kết luận và hướng phát triển	11
A	Phân công và checklist nộp bài	13
B	Hướng dẫn tái lập đầy đủ	14
B.1	Cài môi trường và kiểm thử	14
B.2	Tạo lại kết quả và artefact	14
C	Mã nguồn cốt lõi	15
C.1	Matching và metric instance	15
C.2	Star-convex geometry reference	19
C.3	Script thí nghiệm chạy thật	25
C.4	Runner StarDist ImageJ/Fiji	40

Chương 1

Giới thiệu

1.1 Bối cảnh và động cơ

Ảnh hiển vi thường chứa nhiều nhân tế bào sát nhau. Mask foreground nhị phân chỉ trả lời pixel nào thuộc tế bào, nhưng không trả lời pixel đó thuộc tế bào nào. Vì vậy, một vùng chạm nhau có thể bị gộp thành một object, làm sai số đếm, diện tích và các phép đo hình thái. Instance segmentation yêu cầu một nhãn riêng cho từng object, khác semantic segmentation và object detection chỉ dùng bounding box.

StarDist biểu diễn mỗi object bằng một đa giác star-convex, phù hợp với nhiều nhân tế bào dạng blob. So với cách phân đoạn bottom-up, các proposal hình học và NMS giúp tách object ngay cả khi ranh giới pixel mờ [6]. Tuy nhiên giả thiết star-convex không phù hợp hoàn toàn với object hình vòng, phân nhánh hoặc lõm sâu; đánh giá trên dữ liệu mục tiêu là bắt buộc.

1.2 Mục tiêu và đóng góp

Mục tiêu là nghiên cứu, cài đặt, thực nghiệm và đánh giá StarDist 2D với Fiji làm tham chiếu. Sản phẩm gồm: (i) mô hình toán học của StarDist, CNN/U-Net, Otsu và Watershed; (ii) protocol Fiji có log tham số và macro batch; (iii) package Python với CLI, baseline, geometry reference và metric; (iv) thiết kế thí nghiệm định lượng; và (v) báo cáo, presentation.

1.3 Câu hỏi nghiên cứu

RQ1 kiểm tra StarDist có tách object chạm nhau tốt hơn Otsu/Watershed không. RQ2 kiểm tra tính tương thích Fiji–Python khi giữ nguyên input và tham số. RQ3 đo độ nhạy của probability threshold và NMS threshold. Mỗi câu hỏi dùng test set cố định; validation chỉ dùng để chọn tham số.

Chương 2

Cơ sở lý thuyết

2.1 Bài toán instance segmentation

Cho ảnh $I : \Omega \rightarrow \mathbb{R}^C$, $\Omega = \{0, \dots, H - 1\} \times \{0, \dots, W - 1\}$. Ground truth là ảnh nhãn $Y : \Omega \rightarrow \{0, \dots, M\}$. Nhãn 0 là nền và mỗi nhãn dương là một instance. Mục tiêu là ước lượng $\hat{Y} = f_\theta(I)$, vẫn phân biệt được hai object cùng loại khi chúng tiếp xúc.

2.2 CNN và U-Net

CNN trích đặc trưng cục bộ bằng tích chập. Ở lớp l ,

$$a_{i,j,c}^{(l)} = \phi \left(b_c^{(l)} + \sum_{u,v,c'} w_{u,v,c',c}^{(l)} a_{i+u,j+v,c'}^{(l-1)} \right). \quad (2.1)$$

U-Net gồm encoder thu ngữ cảnh, decoder phục hồi độ phân giải và skip connections giữ chi tiết biên [5]. StarDist dùng chung backbone này cho hai output head thay vì trực tiếp sinh mask cuối.

2.3 Probability map và star-convex polygon

Một tập S star-convex đối với tâm c nếu $[c, x] \subseteq S$ với mọi $x \in S$. Chọn K hướng $\theta_k = 2\pi k/K$; pixel p có vector khoảng cách $\mathbf{r}(p) = [r_0(p), \dots, r_{K-1}(p)]$, trong đó r_k là khoảng cách đến biên object theo tia θ_k . Đỉnh polygon là

$$v_k(p) = p + \hat{r}_k(p)(\cos \theta_k, \sin \theta_k). \quad (2.2)$$

Bài báo gốc dùng $K = 32$. Nhiều tia hơn làm mịn biên nhưng tăng số output và chi phí NMS. Probability target là normalized Euclidean distance transform (EDT) tính riêng trong mỗi instance; đỉnh ở gần tâm, biên bằng 0.

2.4 Huấn luyện và NMS

Mạng học probability $\hat{p}$ và distances $\hat{r}$. Binary cross-entropy phù hợp với head probability; distance loss thường là MAE được trọng số bởi probability target để ưu tiên tâm object. Mỗi pixel foreground tạo một proposal, do đó có nhiều proposal cho cùng object. NMS sắp theo score, giữ proposal cao nhất rồi loại proposal overlap vượt ngưỡng. Hai ngưỡng cần phân biệt: probability threshold quyết định candidate nào được xét, còn NMS threshold quyết định proposal nào bị triệt tiêu.

2.5 Baseline Otsu và Watershed

Otsu chọn ngưỡng t cực đại hoá phương sai giữa hai lớp $\sigma_B^2(t) = \omega_0(t)\omega_1(t)[\mu_0(t) - \mu_1(t)]^2$ [4]. Sau threshold, connected components tạo instance nhưng sẽ gộp object chạm nhau. Watershed dùng local maxima của distance map làm marker rồi “ngập” $-D$ trong foreground mask [8]. Nó có thể tách object chạm nhau, nhưng nhạy với marker, nhiễu và min-distance.

Chương 3

Fiji/ImageJ làm nền tảng tham chiếu

3.1 Plugin StarDist

Plugin StarDist của Fiji cung cấp inference cho model 2D pretrained/custom, percentile normalization, probability threshold, overlap/NMS threshold, tile và output label image/ROI [2]. Fiji là công cụ tham chiếu cho thao tác trực quan và đo đạc, không thay thế đánh giá định lượng.

3.2 Protocol tái lập và lần chạy thực tế

Ngoài protocol GUI, nhóm chạy command StarDist đã công bố `de.csbdresden:StarDist_:0.3.0-scijava` trong ImageJ 2.9.0 headless, với ImageJ 1.53t legacy, Eclipse Temurin 8.0.504 và TensorFlow 1.12 CPU. Đây là chính artifact plugin dùng cho Fiji/ImageJ; Java 8 cần thiết vì TensorFlow ImageJ 1.12 là runtime cũ. Ba ảnh low/median/high được chọn trước từ test split theo AP của StarDist Python, không tune threshold trên ba ảnh này. Model là *Versatile (fluorescent nuclei)*, percentile 1/99.8, probability 0.479071, overlap/NMS 0.3, một tile và boundary exclusion 2. Plugin xuất Label Image TIFF cùng kích thước input. Mỗi lần chạy và runtime được ghi trong `fiji/run_log_dsb2018_reference.csv`; runner Java là `fiji/scripts/FijiStarDistRunner.java`.

File `fiji/workflows/stardist_fiji_protocol.md` vẫn là hướng dẫn thao tác Fiji GUI; macro `fiji/macros/stardist_batch.ijm` yêu cầu dùng Fiji Recorder để chèn đúng lệnh của phiên bản plugin, tránh đoán option macro.

3.3 Kiểm chứng Fiji–Python

Hai output chỉ được so sánh khi dùng cùng ảnh, model, scale, chuẩn hoá và hai threshold. Python đọc trực tiếp label TIFF Fiji rồi ghép instance theo IoU. Nếu số object khác nhau, kiểm tra overlay và run log trước, sau đó mới kết luận do phiên bản/model hay hậu xử lý.

Chương 4

Cài đặt Python

4.1 Kiến trúc phần mềm

Package `src/xla_gr03` tách tiền xử lý, I/O TIFF, Otsu/Watershed, geometry, StarDist wrapper, metrics và CLI. Mọi output là label integer 2D. Lệnh ‘`xla-gr03 segment`’ chạy một phương pháp; ‘`xla-gr03 evaluate`’ xuất JSON metric. Unit tests bao phủ geometry, metric, preprocessing, I/O và baseline.

4.2 Phần nhóm tự cài đặt và phần tái sử dụng

Nhóm tự cài đặt EDT chuẩn hoá theo instance, ray distance, polygon decode, polygon NMS raster hoá, IoU matrix và Hungarian matching. CNN/model pretrained được gọi từ thư viện StarDist chính thức [7]. Sự phân biệt này tránh tuyên bố sai rằng nhóm tự train lại toàn bộ mạng.

Listing 4.1: Ví dụ chạy một baseline

```
xla-gr03 segment data/raw/image_001.tif results/otsu_labels.tif \
--method otsu --foreground bright --min-size 20
xla-gr03 evaluate data/ground_truth/image_001.tif results/otsu_labels
.tif \
--thresholds 0.5 0.75 0.9
```

4.3 Tái lập môi trường

Môi trường đề xuất dùng Python 3.11, NumPy, SciPy, scikit-image, tifffile, pytest và ruff; extra ‘`stardist`’ thêm TensorFlow, csbdeep và StarDist. Chạy ‘`conda env create -f environment.yml`’, sau đó ‘`pytest`’ và ‘`ruff check .`’. Model weight và raw dataset không được commit; config mặc định ở ‘`configs/default.yaml`’.

Chương 5

Thiết kế thí nghiệm và metric

5.1 Dữ liệu và split

Dataset phải có instance ground truth hợp pháp, ví dụ DSB2018 hoặc MoNuSeg. Raw image và label TIFF cùng basename nằm trong ‘data/raw’ và data/ground_truth. Development dùng để debug; validation chọn tham số; test chỉ đánh giá cuối. Không tối ưu threshold trên test set.

5.2 Matching và metric

Gọi G_i, P_j là ground-truth và prediction, $J_{ij} = |G_i \cap P_j|/|G_i \cup P_j|$. Hungarian assignment tạo matching một-một; pair có $J_{ij} \geq \tau$ là TP, object còn lại là FP/FN. Tại mỗi $\tau \in \{0.50, 0.75, 0.90\}$:

$$P = TP/(TP + FP), \quad R = TP/(TP + FN), \quad (5.1)$$

$$F1 = 2TP/(2TP + FP + FN), \quad AP_\tau = TP/(TP + FP + FN). \quad (5.2)$$

Ở đây AP_τ là naming của StarDist, không phải area under PR curve. Panoptic Quality $PQ = \sum J_{ij}/(TP + 0.5FP + 0.5FN)$ [3]. Dice/IoU foreground được báo cáo bổ sung; MAE đếm và runtime trả lời khía cạnh ứng dụng.

5.3 Protocol benchmark

Otsu, Watershed và StarDist Python chạy trên toàn bộ 50 test images. Watershed được chọn bằng validation; StarDist dùng default đã đóng gói. StarDist Fiji/ImageJ chạy trên ba ca low/median/high cố định để kiểm chứng cross-platform với output label thật, không được gộp vào ước lượng 50 ảnh. Báo cáo mean $\pm$ SD, số ảnh và runtime riêng cho từng stratum. Script `scripts/run_dsb2018_benchmark` sinh benchmark 50 ảnh; script `scripts/evaluate_fiji_reference.py` đọc Label Image Fiji rồi đo metric.

Chương 6

Kết quả và thảo luận

6.1 Benchmark tái lập được trên DSB2018

Nhóm chạy ba phương pháp trên 50 ảnh của test split trong archive DSB2018 do StarDist công bố [1]. Ground truth là instance label TIFF; không dùng binary mask. Watershed chọn ‘min_distance’ bằng sweep trên 30 ảnh lấy mẫu seed 42 từ 447 ảnh train, đạt AP@0.50 cao nhất tại giá trị 9. Các 50 ảnh test không tham gia chọn tham số. StarDist Python dùng model 2D_versatile_fluo, percentile 1/99.8, default probability threshold 0.479071 và NMS threshold 0.3 từ model. Runtime CPU không tính tải model và warm-up. Lệnh benchmark Python sinh toàn bộ CSV, figure và provenance; đường dẫn tái lập được liệt kê trong README và Phụ lục C.

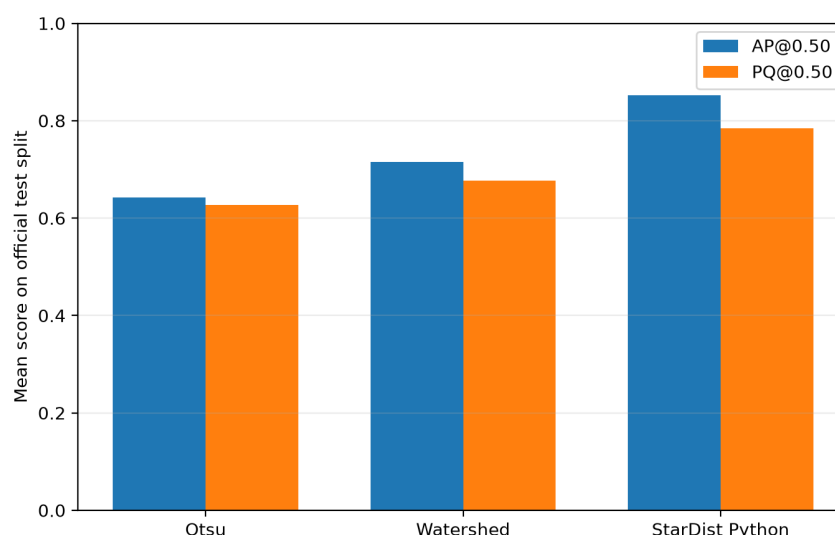
Bảng 6.1: Metric instance. Ba baseline/Python dùng 50 test images; Fiji/ImageJ dùng ba ca kiểm chứng cố định.

Method	AP@.50	AP@.75	AP@.90	PQ@.50
Otsu	0.643 $\pm$ 0.238	0.453 $\pm$ 0.276	0.223 $\pm$ 0.221	0.627 $\pm$ 0.208
Watershed	0.715 $\pm$ 0.199	0.501 $\pm$ 0.266	0.214 $\pm$ 0.207	0.677 $\pm$ 0.166
StarDist Fiji ($n = 3$)	0.801 $\pm$ 0.244	0.764 $\pm$ 0.207	0.406 $\pm$ 0.324	0.788 $\pm$ 0.160
StarDist Python	0.853 $\pm$ 0.120	0.692 $\pm$ 0.235	0.305 $\pm$ 0.313	0.785 $\pm$ 0.110

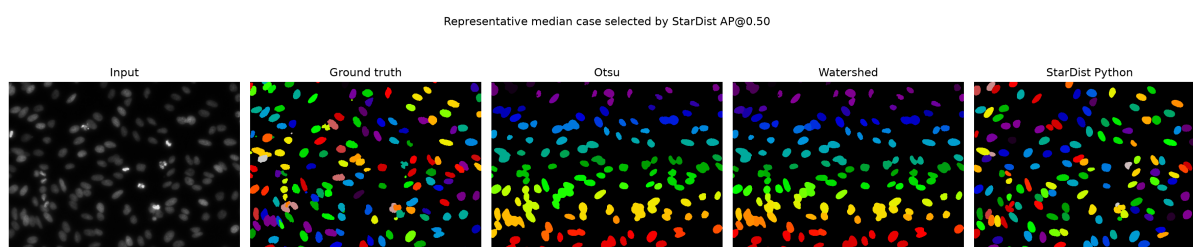
Bảng 6.2: Metric foreground, sai số đếm và runtime. Mỗi dòng Fiji là mean $\pm$ SD trên ba ca.

Method	Dice foreground	MAE count	Runtime (s/image)
Otsu	0.895 ± 0.070	14.06	0.0100
Watershed	0.894 ± 0.071	6.94	0.0238
StarDist Fiji ($n = 3$)	0.950 ± 0.017	6.00 ± 7.21	1.420 ± 0.590
StarDist Python	0.916 ± 0.049	3.44	0.1120

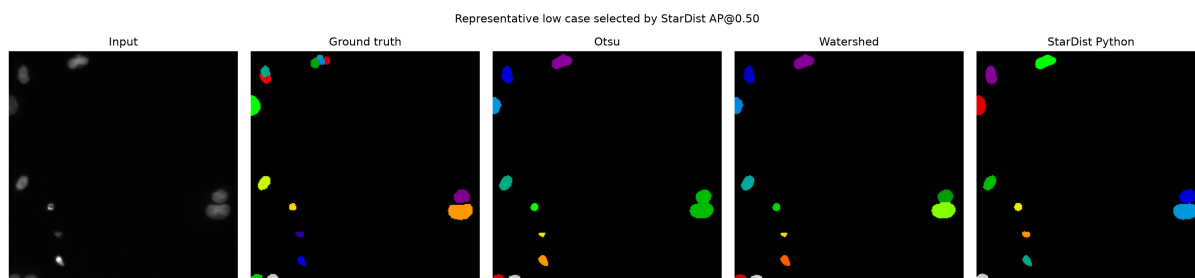
Ghi chú về công bằng so sánh. Dòng Fiji/ImageJ là kiểm chứng triển khai trên ba ca đã cố định; không phải ước lượng chung cho 50 test images nên không dùng để xếp hạng trực tiếp với ba dòng $n = 50$. Runtime Fiji đo command CPU, bao gồm inference/model initialization của process Java; runtime Python là inference sau warm-up.



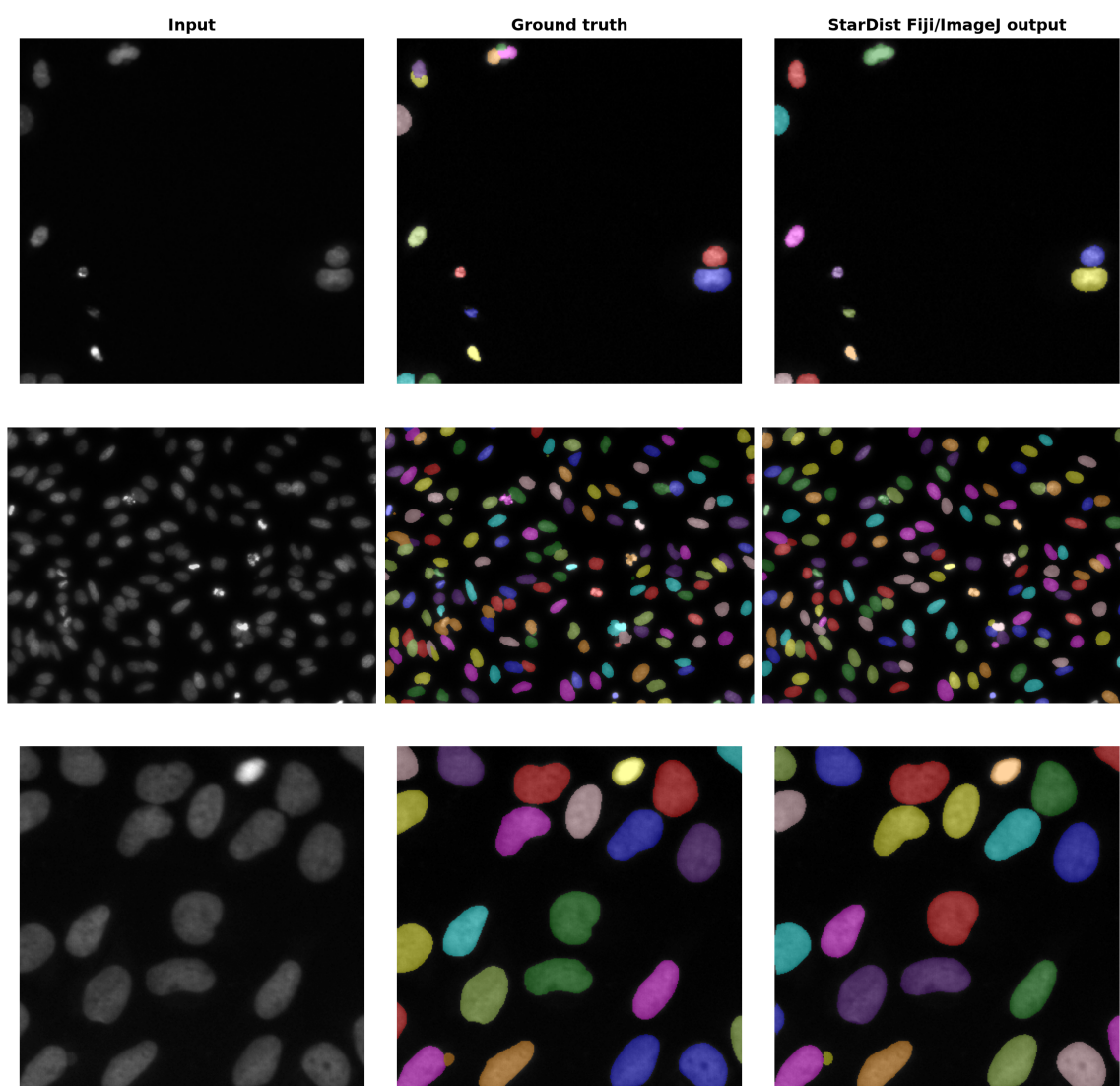
Hình 6.1: Mean AP@0.50 và PQ@0.50 trên 50 ảnh test DSB2018. Error bar được trình bày trong Bảng 6.1.



Hình 6.2: Case trung vị theo AP@0.50 của StarDist trong 50 ảnh test. Màu biểu diễn ID instance, không phải lớp semantic. Otsu gộp nhiều nhân tiếp xúc; Watershed tách được một phần; StarDist bám theo instance ground truth sát hơn.



Hình 6.3: Case có AP@0.50 thấp nhất của StarDist trong test split. Hình này được báo cáo cùng case trung vị để tránh chỉ chọn ví dụ thuận lợi.



Hình 6.4: Ba output Label Image thực tế của plugin StarDist Fiji/ImageJ: ca low, median và high đã chọn trước, đặt cạnh input và ground truth. Mỗi màu là một ID instance; figure được sinh trực tiếp từ TIFF plugin xuất.

6.2 Cách đọc kết quả

Trên 50 ảnh test, StarDist đạt $AP@0.50\ 0.853 \pm 0.120$ và $PQ\ 0.785 \pm 0.110$, cao hơn Watershed (0.715 ± 0.199 ; 0.677 ± 0.166) và Otsu (0.643 ± 0.238 ; 0.627 ± 0.208). Dice foreground của Otsu và Watershed gần 0.895, nhưng AP instance thấp hơn: kết quả này cho thấy Dice toàn foreground không đủ để đánh giá tách object. Watershed giảm MAE đếm từ 14.06 xuống 6.94 so với Otsu; StarDist giảm tiếp còn 3.44. Ở $\tau = 0.90$, metric của cả ba giảm mạnh, phản ánh yêu cầu chính xác biên rất nghiêm ngặt. StarDist chậm hơn baseline trên CPU (0.112 s/ảnh so với 0.024 s Watershed), đổi lại tách instance tốt hơn trên benchmark này. Trên ba ca kiểm chứng, plugin Fiji/ImageJ đạt $AP@0.50\ 0.801 \pm 0.244$, $PQ\ 0.788 \pm 0.160$ và $Dice\ 0.950 \pm 0.017$; các TIFF/metric làm bằng chứng tương thích, không thay thế benchmark $n = 50$.

6.3 Đo đạc tới tính hợp lệ

Model pretrained có thể không phù hợp stain/optics của dữ liệu. Khác biệt Fiji và Python có thể do version, normalization, tile hoặc NMS, không nhất thiết do thuật toán. Dataset nhỏ dẫn tới SD lớn. Instance có hình không star-convex là giới hạn cấu trúc; fine-tuning hoặc phương pháp khác cần được cân nhắc.

Chương 7

Kết luận và hướng phát triển

StarDist biến dự đoán pixel dày đặc thành tập polygon có định danh instance, phù hợp với ảnh nhân tế bào đông đúc. Repo cung cấp pipeline kiểm chứng được giữa Fiji/ImageJ và Python, baseline cổ điển, metric instance, label TIFF thực tế và run log. Hướng phát triển là fine-tuning bằng annotation nội bộ, ablation số tia, robustness với noise và đánh giá 3D.

Tài liệu tham khảo

- [1] Juan C. Caicedo, Allen Goodman, Kyle W. Karhohs, et al. Nucleus segmentation across imaging experiments: the 2018 data science bowl. *Nature Methods*, 16(12):1247–1253, 2019. doi: 10.1038/s41592-019-0612-7.
- [2] ImageJ. Stardist. <https://imagej.net/plugins/stardist>, 2026. Accessed: 2026-10-03.
- [3] Alexander Kirillov, Kaiming He, Ross Girshick, Carsten Rother, and Piotr Dollár. Panoptic segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9404–9413, 2019. doi: 10.1109/CVPR.2019.00963.
- [4] Nobuyuki Otsu. A threshold selection method from gray-level histograms. *IEEE Transactions on Systems, Man, and Cybernetics*, 9(1):62–66, 1979. doi: 10.1109/TSMC.1979.4310076.
- [5] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, pages 234–241. Springer, 2015. doi: 10.1007/978-3-319-24574-4_28.
- [6] Uwe Schmidt, Martin Weigert, Coleman Broaddus, and Gene Myers. Cell detection with star-convex polygons. In *Medical Image Computing and Computer Assisted Intervention – MICCAI 2018*, pages 265–273. Springer, 2018. doi: 10.1007/978-3-030-00934-2_30.
- [7] StarDist contributors. Stardist – object detection with star-convex shapes. <https://github.com/stardist/stardist>, 2026. Accessed: 2026-10-03.
- [8] Luc Vincent and Pierre Soille. Watersheds in digital spaces: An efficient algorithm based on immersion simulations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 13(6):583–598, 1991. doi: 10.1109/34.87344.

Phụ lục A

Phân công và checklist nộp bài

Thành viên	Công việc và artefact
Thành viên 1	Lý thuyết, mô hình toán, tài liệu: <code>docs/01_ly_thuyet_stardist.md</code> .
Thành viên 2	Package Python, test, CLI và <code>scripts/run_experiment.py</code> .
Thành viên 3	Fiji plugin, screenshot, macro, run log và label TIFF.
Thành viên 4	Dataset, split, ground truth, bảng/biểu đồ và phân tích.

Trước khi nộp LMS: thay tên/mã nhóm, chốt dataset và license, chạy test/lint, điền bảng số liệu thật, thêm 4–6 hình minh hoạ/biểu đồ từ test set, xuất PDF và kiểm tra số trang, cập nhật slide theo bảng cuối.

Phụ lục B

Hướng dẫn tái lập đầy đủ

B.1 Cài môi trường và kiểm thử

```
python -m venv .venv
.venv\Scripts\python.exe -m pip install -e "[dev,stardist]"
.venv\Scripts\python.exe -m pytest -q
.venv\Scripts\ruff.exe check src tests scripts
```

B.2 Tạo lại kết quả và artefact

```
.venv\Scripts\python.exe scripts/run_real_demo.py
```

Lệnh benchmark chính là ‘scripts/run_dsb2018_benchmark.py’; nó tải archive DSB2018 và model ‘2D_versatile_fluo’ qua API chính thức khi chưa có cache, lưu CSV, label TIFF minh hoạ và figures. Lần chạy đã ghi trong báo cáo dùng CPU Windows; TensorFlow native Windows từ 2.11 không dùng CUDA, nên benchmark GPU phải ghi môi trường WSL2/Linux riêng. Phần ImageJ/Fiji tham chiếu đã lưu label TIFF, figure, metric và run log trên ba ca. Trước nộp chỉ cần điền tên/MSSV/mã nhóm rồi xuất PDF từ nguồn này.

Phụ lục C

Mã nguồn cốt lõi

C.1 Matching và metric instance

Listing C.1: ‘metrics.py’: IoU matrix, Hungarian matching và các metric.

```
"""Quantitative metrics for 2D instance segmentation."""

from __future__ import annotations

from dataclasses import asdict, dataclass

import numpy as np
from numpy.typing import ArrayLike, NDArray
from scipy.optimize import linear_sum_assignment

@dataclass(frozen=True)
class InstanceMetrics:
    """Detection and segmentation statistics at one IoU threshold."""

    threshold: float
    true_objects: int
    predicted_objects: int
    true_positives: int
    false_positives: int
    false_negatives: int
    precision: float
    recall: float
    f1: float
    accuracy: float
    mean_matched_iou: float
```

```

panoptic_quality: float
foreground_dice: float
foreground_iou: float
count_absolute_error: int

def to_dict(self) -> dict[str, float | int]:
    """Return a JSON-serializable representation."""

    return asdict(self)

def _validate_label_pair(
    ground_truth: ArrayLike,
    prediction: ArrayLike,
) -> tuple[NDArray[np.integer], NDArray[np.integer]]:
    true = np.asarray(ground_truth)
    pred = np.asarray(prediction)
    if true.shape != pred.shape or true.ndim != 2:
        raise ValueError("Ground truth and prediction must be 2D arrays with identical shape.")
    for name, array in (("ground_truth", true), ("prediction", pred)):
        if not np.issubdtype(array.dtype, np.integer) or np.any(array < 0):
            raise ValueError(f"{name} must contain non-negative integer labels.")
    return true, pred

def relabel_sequential(labels: ArrayLike) -> NDArray[np.int32]:
    """Map arbitrary positive label IDs to the sequence 1..N."""

    array = np.asarray(labels)
    output = np.zeros(array.shape, dtype=np.int32)
    for new_id, old_id in enumerate(np.unique(array[array > 0]), start=1):
        output[array == old_id] = new_id
    return output

def intersection_over_union_matrix(

```

```

    ground_truth: ArrayLike,
    prediction: ArrayLike,
) -> NDArray[np.float64]:
    """Return pairwise IoU for all positive ground-truth/prediction
       instances."""

    true, pred = _validate_label_pair(ground_truth, prediction)
    true = relabel_sequential(true)
    pred = relabel_sequential(pred)
    n_true = int(true.max(initial=0))
    n_pred = int(pred.max(initial=0))
    if n_true == 0 or n_pred == 0:
        return np.zeros((n_true, n_pred), dtype=np.float64)

    flat_true = true.ravel()
    flat_pred = pred.ravel()
    overlap = np.zeros((n_true + 1, n_pred + 1), dtype=np.int64)
    np.add.at(overlap, (flat_true, flat_pred), 1)
    intersection = overlap[1:, 1:].astype(np.float64)
    true_area = overlap[1:, :].sum(axis=1, keepdims=True)
    pred_area = overlap[:, 1:].sum(axis=0, keepdims=True)
    union = true_area + pred_area - intersection
    return np.divide(intersection, union, out=np.zeros_like(
        intersection), where=union > 0)

def _safe_ratio(numerator: float, denominator: float) -> float:
    return float(numerator / denominator) if denominator else 0.0

def evaluate_instances(
    ground_truth: ArrayLike,
    prediction: ArrayLike,
    threshold: float = 0.5,
) -> InstanceMetrics:
    """Evaluate a prediction using one-to-one IoU matching.

    The assignment maximizes the number of pairs at or above ``
    threshold`` and
    uses IoU as a tie-breaker, following the StarDist matching
    implementation.

```

```
"""
```

```
if not 0 <= threshold <= 1:
    raise ValueError("threshold must be in [0,1].")
true, pred = _validate_label_pair(ground_truth, prediction)
iou = intersection_over_union_matrix(true, pred)
n_true, n_pred = iou.shape
matched_iou = np.array([], dtype=np.float64)

if n_true and n_pred:
    n_possible = min(n_true, n_pred)
    cost = -(iou >= threshold).astype(np.float64) - iou / (2.0 *
        n_possible)
    true_indices, pred_indices = linear_sum_assignment(cost)
    assigned_iou = iou[true_indices, pred_indices]
    matched_iou = assigned_iou[assigned_iou >= threshold]

true_positives = int(matched_iou.size)
false_positives = n_pred - true_positives
false_negatives = n_true - true_positives
precision = _safe_ratio(true_positives, true_positives +
    false_positives)
recall = _safe_ratio(true_positives, true_positives +
    false_negatives)
f1 = _safe_ratio(2 * true_positives, 2 * true_positives +
    false_positives + false_negatives)
accuracy = _safe_ratio(
    true_positives, true_positives + false_positives +
    false_negatives
)
mean_matched_iou = float(matched_iou.mean()) if true_positives
    else 0.0
pq_denominator = true_positives + 0.5 * false_positives + 0.5 *
    false_negatives
panoptic_quality = _safe_ratio(float(matched_iou.sum()),
    pq_denominator)

true_foreground = true > 0
pred_foreground = pred > 0
intersection = int(np.logical_and(true_foreground,
    pred_foreground).sum())
```

```

true_area = int(true_foreground.sum())
pred_area = int(pred_foreground.sum())
foreground_dice = _safe_ratio(2 * intersection, true_area +
    pred_area)
foreground_iou = _safe_ratio(intersection, true_area + pred_area
    - intersection)

return InstanceMetrics(
    threshold=float(threshold),
    true_objects=n_true,
    predicted_objects=n_pred,
    true_positives=true_positives,
    false_positives=false_positives,
    false_negatives=false_negatives,
    precision=precision,
    recall=recall,
    f1=f1,
    accuracy=accuracy,
    mean_matched_iou=mean_matched_iou,
    panoptic_quality=panoptic_quality,
    foreground_dice=foreground_dice,
    foreground_iou=foreground_iou,
    count_absolute_error=abs(n_pred - n_true),
)

def evaluate_at_thresholds(
    ground_truth: ArrayLike,
    prediction: ArrayLike,
    thresholds: list[float] | tuple[float, ...],
) -> list[InstanceMetrics]:
    """Evaluate the same label pair at multiple IoU thresholds."""

    if not thresholds:
        raise ValueError("At least one threshold is required.")
    return [evaluate_instances(ground_truth, prediction, value) for
        value in thresholds]

```

C.2 Star-convex geometry reference

Listing C.2: ‘geometry.py’: EDT, ray distance, polygon decode và NMS.

```

"""Reference implementations of StarDist geometric building blocks.

These functions prioritize readability and verifiability over raw
speed. They are
intended for coursework demonstrations and unit tests, not full model
training.
"""

from __future__ import annotations

from collections.abc import Sequence

import numpy as np
from numpy.typing import ArrayLike, NDArray
from scipy import ndimage as ndi
from skimage.draw import polygon as rasterize_polygon

def _validate_labels(labels: ArrayLike) -> NDArray[np.integer]:
    array = np.asarray(labels)
    if array.ndim != 2 or not np.issubdtype(array.dtype, np.integer):
        raise ValueError("Instance labels must be a 2D integer array.
        ")
    if np.any(array < 0):
        raise ValueError("Instance labels must be non-negative.")
    return array

def normalized_edt(labels: ArrayLike) -> NDArray[np.float32]:
    """Compute an Euclidean distance target normalized per object
    instance.

    Background is zero. Every positive object is transformed
    independently and
    divided by its own maximum, matching the definition used by
    StarDist.
    """

    label_array = _validate_labels(labels)
    probability = np.zeros(label_array.shape, dtype=np.float32)

```

```

for object_id in np.unique(label_array):
    if object_id == 0:
        continue
    mask = label_array == object_id
    padded = np.pad(mask, 1, mode="constant", constant_values=
        False)
    distance = ndi.distance_transform_edt(padded)[1:-1, 1:-1]
    maximum = float(distance[mask].max(initial=0.0))
    if maximum > 0:
        probability[mask] = distance[mask] / maximum
return probability

def ray_angles(n_rays: int) -> NDArray[np.float64]:
    """Return ``n_rays`` equally spaced angles in ``[0, 2*pi)``."""

    if n_rays < 3:
        raise ValueError("n_rays must be at least 3.")
    return np.linspace(0.0, 2.0 * np.pi, n_rays, endpoint=False)

def star_distances(
    labels: ArrayLike,
    n_rays: int = 32,
    step: float = 1.0,
) -> NDArray[np.float32]:
    """Estimate radial distances from every foreground pixel to its
        boundary.

        The implementation walks along equally spaced rays until the
        nearest sampled
        pixel no longer has the same instance ID. It is intentionally
        simple and slow,
        so use small masks to explain or validate the StarDist target
        representation.
    """

    label_array = _validate_labels(labels)
    if step <= 0:
        raise ValueError("step must be positive.")

```

```

height, width = label_array.shape
max_distance = float(np.hypot(height, width) + step)
angles = ray_angles(n_rays)
directions = np.column_stack((np.sin(angles), np.cos(angles)))
distances = np.zeros((height, width, n_rays), dtype=np.float32)

for y0, x0 in np.argwhere(label_array > 0):
    object_id = label_array[y0, x0]
    for ray_index, (dy, dx) in enumerate(directions):
        distance = step
        while distance <= max_distance:
            y = int(np rint(y0 + distance * dy))
            x = int(np rint(x0 + distance * dx))
            if y < 0 or y >= height or x < 0 or x >= width:
                break
            if label_array[y, x] != object_id:
                break
            distance += step
        distances[y0, x0, ray_index] = distance
return distances

def decode_polygon(
    center_yx: Sequence[float],
    distances: ArrayLike,
) -> NDArray[np.float64]:
    """Decode radial distances into polygon vertices in `(x, y)` order."""

    center = np.asarray(center_yx, dtype=np.float64)
    radii = np.asarray(distances, dtype=np.float64)
    if center.shape != (2,) or radii.ndim != 1 or radii.size < 3:
        raise ValueError("Expected center_yx=(y, x) and a 1D vector with at least 3 radii.")
    if np.any(radii < 0):
        raise ValueError("Polygon radii must be non-negative.")

    y0, x0 = center
    angles = ray_angles(radii.size)
    x = x0 + radii * np.cos(angles)
    y = y0 + radii * np.sin(angles)

```

```

    return np.column_stack((x, y))

def polygon_area(polygon_xy: ArrayLike) -> float:
    """Calculate polygon area with the shoelace formula."""

    polygon = np.asarray(polygon_xy, dtype=np.float64)
    if polygon.ndim != 2 or polygon.shape[1] != 2 or polygon.shape[0]
        < 3:
        raise ValueError("A polygon must have shape (N, 2) with N >= 3.")
    x, y = polygon[:, 0], polygon[:, 1]
    return float(0.5 * abs(np.dot(x, np.roll(y, -1)) - np.dot(y, np.
        roll(x, -1))))

def polygon_overlap_ratio(
    polygon_a_xy: ArrayLike,
    polygon_b_xy: ArrayLike,
    mode: str = "smaller",
) -> float:
    """Estimate polygon overlap after rasterization on a shared local
    canvas.

    ``mode='smaller'`` returns intersection divided by the smaller
    polygon area,
    which mirrors StarDist 2D NMS. ``mode='iou'`` returns the
    standard IoU.
    """

    polygon_a = np.asarray(polygon_a_xy, dtype=np.float64)
    polygon_b = np.asarray(polygon_b_xy, dtype=np.float64)
    if mode not in {"smaller", "iou"}:
        raise ValueError("mode must be either 'smaller' or 'iou'.")
    for polygon in (polygon_a, polygon_b):
        if polygon.ndim != 2 or polygon.shape[1] != 2 or polygon.
            shape[0] < 3:
            raise ValueError("Every polygon must have shape (N, 2)
                with N >= 3.")

    all_points = np.vstack((polygon_a, polygon_b))

```

```

min_x, min_y = np.floor(all_points.min(axis=0)).astype(int) - 1
max_x, max_y = np.ceil(all_points.max(axis=0)).astype(int) + 1
width = max_x - min_x + 1
height = max_y - min_y + 1
if width <= 0 or height <= 0:
    return 0.0

masks: list[NDArray[np.bool_]] = []
for polygon in (polygon_a, polygon_b):
    shifted_x = polygon[:, 0] - min_x
    shifted_y = polygon[:, 1] - min_y
    rows, cols = rasterize_polygon(shifted_y, shifted_x, shape=(
        height, width))
    mask = np.zeros((height, width), dtype=bool)
    mask[rows, cols] = True
    masks.append(mask)

area_a = int(masks[0].sum())
area_b = int(masks[1].sum())
intersection = int(np.logical_and(masks[0], masks[1]).sum())
if mode == "smaller":
    denominator = min(area_a, area_b)
else:
    denominator = area_a + area_b - intersection
return intersection / denominator if denominator else 0.0

def greedy_polygon_nms(
    polygons_xy: Sequence[ArrayLike],
    scores: ArrayLike,
    threshold: float = 0.3,
    overlap_mode: str = "smaller",
) -> NDArray[np.int64]:
    """Return indices retained by score-sorted greedy polygon NMS."""

    if not 0 <= threshold <= 1:
        raise ValueError("threshold must be in [0, 1].")
    score_array = np.asarray(scores, dtype=np.float64)
    if score_array.ndim != 1 or score_array.size != len(polygons_xy):
        raise ValueError("scores must contain one value for every
            polygon.")

```

```

order = np.argsort(-score_array, kind="stable")
kept: list[int] = []
for candidate in order:
    if all(
        polygon_overlap_ratio(
            polygons_xy[int(candidate)], polygons_xy[selected],
            mode=overlap_mode
        )
        <= threshold
        for selected in kept
    ):
        kept.append(int(candidate))
return np.asarray(kept, dtype=np.int64)

```

C.3 Script thí nghiệm chạy thật

Listing C.3: ‘run_real_demo.py’: materialize sample, inference, evaluation và figure.

```

"""Run a real, reproducible StarDist demonstration on its DSB2018
sample.

The official StarDist package distributes this fluorescence image and
instance
mask as a sample from the 2018 Data Science Bowl. This script
materializes the
sample locally, runs the three methods, exports TIFF labels/CSV/
figures, and
marks the output as a one-image demonstration rather than a benchmark
.
"""

from __future__ import annotations

import csv
import json
import time
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np

```

```

from stardist.data import test_image_nuclei_2d
from stardist.models import StarDist2D
from tifffile import imwrite

from xla_gr03.baselines import segment_otsu, segment_watershed
from xla_gr03.io import save_labels
from xla_gr03.metrics import evaluate_at_thresholds,
    evaluate_instances
from xla_gr03.preprocessing import percentile_normalize

ROOT = Path(__file__).resolve().parents[1]
RAW = ROOT / "data" / "raw" / "dsb2018_stardist_sample.tif"
GT = ROOT / "data" / "ground_truth" / "dsb2018_stardist_sample.tif"
OUT = ROOT / "results" / "real_demo"
LABELS = OUT / "labels"
FIGURES = OUT / "figures"
THRESHOLDS = (0.5, 0.75, 0.9)

def write_csv(path: Path, rows: list[dict[str, object]]) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    fields = list(dict.fromkeys(key for row in rows for key in row))
    with path.open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=fields,
                                extrasaction="ignore")
        writer.writeheader()
        writer.writerows(rows)

def save_panel(image: np.ndarray, truth: np.ndarray, predictions:
dict[str, np.ndarray]) -> None:
    FIGURES.mkdir(parents=True, exist_ok=True)
    panels = [("Input_fluorescence", image), ("Ground_truth", truth),
               *predictions.items()]
    figure, axes = plt.subplots(1, len(panels), figsize=(4 * len(
        panels), 4))
    for axis, (name, data) in zip(axes, panels, strict=True):
        axis.imshow(data, cmap="gray" if name == "Input_fluorescence"
                     else "nipy_spectral")
        axis.set_title(name)
        axis.axis("off")

```

```

figure.tight_layout()
figure.savefig(FIGURES / "qualitative_comparison.png", dpi=200,
               bbox_inches="tight")
plt.close(figure)

def save_sweep_plot(rows: list[dict[str, object]], value: str,
                    filename: str) -> None:
    figure, axis = plt.subplots(figsize=(6, 4))
    for method, marker in (("probability_sweep", "o"), ("nms_sweep",
                                                         "s")):
        subset = [row for row in rows if row["sweep"] == method]
        x_key = "probability_threshold" if method == "
            probability_sweep" else "nms_threshold"
        axis.plot(
            [float(row[x_key]) for row in subset],
            [float(row[value]) for row in subset],
            marker=marker,
            label=method.replace("_", "␣"),
        )
    axis.set_xlabel("Threshold")
    axis.set_ylabel(value.replace("_", "␣"))
    axis.grid(alpha=0.25)
    axis.legend()
    figure.tight_layout()
    figure.savefig(FIGURES / filename, dpi=200)
    plt.close(figure)

def predict(
    model: StarDist2D,
    image: np.ndarray,
    probability_threshold: float,
    nms_threshold: float,
) -> tuple[np.ndarray, float]:
    normalized = percentile_normalize(image, pmin=1.0, pmax=99.8,
                                     axes=(0, 1))
    start = time.perf_counter()
    labels, _ = model.predict_instances(
        normalized,
        axes="YX",
    )

```

```

        prob_thresh=probability_threshold,
        nms_thresh=nms_threshold,
    )
    return np.asarray(labels, dtype=np.int32), time.perf_counter() -
        start

def main() -> None:
    np.random.seed(42)
    image, truth = test_image_nuclei_2d(return_mask=True)
    RAW.parent.mkdir(parents=True, exist_ok=True)
    GT.parent.mkdir(parents=True, exist_ok=True)
    OUT.mkdir(parents=True, exist_ok=True)
    imwrite(RAW, image)
    imwrite(GT, truth.astype(np.uint16))

    methods: dict[str, tuple[np.ndarray, float, dict[str, object]]] =
        {}
    for name, method in (
        ("otsu", lambda: segment_otsu(image, foreground="bright",
            min_size=20)),
        (
            "watershed",
            lambda: segment_watershed(
                image, foreground="bright", min_size=20, min_distance
                =7
            ),
        ),
    ):
        start = time.perf_counter()
        result = method()
        methods[name] = (result.labels, time.perf_counter() - start,
            result.metadata)

    model = StarDist2D.from_pretrained("2D_versatile_fluo")
    # Warm-up downloads/builds the model before the recorded
    inference.
    predict(model, image, probability_threshold=0.5, nms_threshold
        =0.3)
    labels, elapsed = predict(model, image, probability_threshold
        =0.5, nms_threshold=0.3)

```

```

methods["stardist_python"] = (
    labels,
    elapsed,
    {
        "model": "2D_versatile_fluo",
        "pmin": 1.0,
        "pmax": 99.8,
        "probability_threshold": 0.5,
        "nms_threshold": 0.3,
        "device": "CPU",
    },
)

rows: list[dict[str, object]] = []
panels: dict[str, np.ndarray] = {}
for name, (prediction, elapsed, metadata) in methods.items():
    save_labels(LABELS / f"{name}.tif", prediction)
    panels[name] = prediction
    for metric in evaluate_at_thresholds(truth, prediction,
        THRESHOLDS):
        rows.append(
            {
                "method": name,
                "runtime_s": round(elapsed, 6),
                "source": "official_StarDist_DSB2018_sample",
                **metadata,
                **metric.to_dict(),
            }
        )
write_csv(OUT / "metrics_per_threshold.csv", rows)
save_panel(image, truth, panels)

sweep_rows: list[dict[str, object]] = []
for probability_threshold in (0.3, 0.4, 0.5, 0.6, 0.7):
    prediction, elapsed = predict(model, image,
        probability_threshold, 0.3)
    metric = evaluate_instances(truth, prediction, threshold=0.5)
    sweep_rows.append(
        {
            "sweep": "probability_sweep",
            "probability_threshold": probability_threshold,

```

```

        "nms_threshold": 0.3,
        "runtime_s": round(elapsed, 6),
        **metric.to_dict(),
    }
)
for nms_threshold in (0.1, 0.2, 0.3, 0.4, 0.5):
    prediction, elapsed = predict(model, image, 0.5,
        nms_threshold)
    metric = evaluate_instances(truth, prediction, threshold=0.5)
    sweep_rows.append(
        {
            "sweep": "nms_sweep",
            "probability_threshold": 0.5,
            "nms_threshold": nms_threshold,
            "runtime_s": round(elapsed, 6),
            **metric.to_dict(),
        }
    )
write_csv(OUT / "stardist_threshold_sweep.csv", sweep_rows)
save_sweep_plot(sweep_rows, "accuracy", "threshold_vs_ap050.png")
save_sweep_plot(sweep_rows, "predicted_objects", "
    threshold_vs_object_count.png")
(OUT / "provenance.json").write_text(
    json.dumps(
        {
            "dataset": "StarDist_distributed_DSB2018_fluorescence
                _sample",
            "source": "https://github.com/stardist/stardist",
            "sample_count": 1,
            "caveat": "Demonstration_only._This_single_image_is_
                not_an_independent_test_set.",
            "model": "2D_versatile_fluo",
            "random_seed": 42,
        },
        ensure_ascii=False,
        indent=2,
    )
    + "\n",
    encoding="utf-8",
)

```

```

if __name__ == "__main__":
    main()

```

Listing C.4: ‘run_dsb2018_benchmark.py’: validation, benchmark test split và tổng hợp kết quả.

```

"""Reproduce a multi-image DSB2018 benchmark for StarDist and two
    baselines.

    The archive is the official split distributed by the StarDist project.
    The training partition selects the marker spacing for Watershed; the
    separate test partition reports the final metrics. The pretrained StarDist model
    runs with its published default thresholds, without tuning on this benchmark's
    test set.
"""

from __future__ import annotations

import csv
import json
import statistics
import time
import urllib.request
import zipfile
from collections import defaultdict
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
from stardist.models import StarDist2D
from tifffile import imread

from xla_gr03.baselines import segment_otsu, segment_watershed
from xla_gr03.io import save_labels
from xla_gr03.metrics import evaluate_at_thresholds,
    evaluate_instances
from xla_gr03.preprocessing import percentile_normalize

ROOT = Path(__file__).resolve().parents[1]

```

```

DATA = ROOT / "data" / "dsb2018"
ARCHIVE = DATA / "dsb2018.zip"
DATASET_ROOT = DATA / "dsb2018"
RESULTS = ROOT / "results" / "dsb2018_benchmark"
URL = "https://github.com/stardist/stardist/releases/download/0.1.0/
      dsb2018.zip"
IOU_THRESHOLDS = (0.5, 0.75, 0.9)
VALIDATION_MIN_DISTANCE = (3, 5, 7, 9, 11)

def write_csv(path: Path, rows: list[dict[str, object]]) -> None:
    fields = list(dict.fromkeys(key for row in rows for key in row))
    with path.open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=fields)
        writer.writeheader()
        writer.writerows(rows)

def ensure_dataset() -> None:
    test_dir = DATASET_ROOT / "test" / "images"
    if test_dir.is_dir() and any(test_dir.glob("*.tif")):
        return
    DATA.mkdir(parents=True, exist_ok=True)
    if not ARCHIVE.exists():
        print(f"Downloading_{URL}")
        urllib.request.urlretrieve(URL, ARCHIVE)
    with zipfile.ZipFile(ARCHIVE) as archive:
        archive.extractall(DATA)
    if not test_dir.is_dir() or not any(test_dir.glob("*.tif")):
        raise RuntimeError("The_official_archive_did_not_contain_test
                           _image_TIFF_files.")

def paired_files(split: str) -> list[tuple[Path, Path]]:
    image_dir = DATASET_ROOT / split / "images"
    mask_dir = DATASET_ROOT / split / "masks"
    pairs = [(path, mask_dir / path.name) for path in sorted(
        image_dir.glob("*.tif"))]
    missing = [mask for _, mask in pairs if not mask.is_file()]
    if missing:
        raise RuntimeError(f"Missing_{len(missing)}_masks_in_{

```

```

        mask_dir}")
    if not pairs:
        raise RuntimeError(f"No_TIFF_pairs_found_in_{image_dir}")
    return pairs

def choose_watershed_distance(
    validation: list[tuple[Path, Path]],
) -> tuple[int, list[dict[str, object]]]:
    rows: list[dict[str, object]] = []
    for distance in VALIDATION_MIN_DISTANCE:
        scores: list[float] = []
        for image_path, mask_path in validation:
            prediction = segment_watershed(
                imread(image_path), foreground="bright", min_size=20,
                min_distance=distance
            ).labels
            scores.append(evaluate_instances(imread(mask_path),
                prediction, threshold=0.5).accuracy)
        rows.append(
            {
                "min_distance": distance,
                "n_validation": len(scores),
                "ap_iou_050_mean": float(statistics.mean(scores)),
                "ap_iou_050_sd": float(statistics.stdev(scores)) if
                    len(scores) > 1 else 0.0,
            }
        )
    best = max(rows, key=lambda row: (float(row["ap_iou_050_mean"]),
        -int(row["min_distance"])))
    return int(best["min_distance"]), rows

def predict_stardist(
    model: StarDist2D, image: np.ndarray
) -> tuple[np.ndarray, float, float, float]:
    normalized = percentile_normalize(image, pmin=1.0, pmax=99.8,
        axes=(0, 1))
    probability_threshold = float(model.thresholds.prob)
    nms_threshold = float(model.thresholds.nms)
    start = time.perf_counter()

```

```

labels, _ = model.predict_instances(
    normalized,
    axes="YX",
    prob_thresh=probability_threshold,
    nms_thresh=nms_threshold,
)
return (
    np.asarray(labels, dtype=np.int32),
    time.perf_counter() - start,
    probability_threshold,
    nms_threshold,
)

def make_summary(rows: list[dict[str, object]]) -> list[dict[str,
object]]:
    grouped: dict[tuple[str, float], list[dict[str, object]]] =
        defaultdict(list)
    for row in rows:
        grouped[(str(row["method"]), float(row["threshold"]))].append
            (row)
    summary: list[dict[str, object]] = []
    for (method, threshold), values in sorted(grouped.items()):
        numbers_by_metric = {
            key: [float(value[key]) for value in values]
            for key in (
                "accuracy",
                "panoptic_quality",
                "foreground_dice",
                "count_absolute_error",
                "runtime_s",
            )
        }

    def avg(key: str, numbers=numbers_by_metric) -> float:
        return float(statistics.mean(numbers[key]))

    def sd(key: str, numbers=numbers_by_metric) -> float:
        return float(statistics.stdev(numbers[key])) if len(
            numbers[key]) > 1 else 0.0

```

```

summary.append(
    {
        "method": method,
        "threshold": threshold,
        "n_test": len(values),
        "accuracy_mean": avg("accuracy"),
        "accuracy_sd": sd("accuracy"),
        "panoptic_quality_mean": avg("panoptic_quality"),
        "panoptic_quality_sd": sd("panoptic_quality"),
        "foreground_dice_mean": avg("foreground_dice"),
        "foreground_dice_sd": sd("foreground_dice"),
        "count_absolute_error_mean": avg("
            count_absolute_error"),
        "runtime_s_mean": avg("runtime_s"),
    }
)
return summary

def make_figure(summary: list[dict[str, object]]) -> None:
    rows_at_threshold = {
        str(row["method"]): row for row in summary if float(row["
            threshold"]) == 0.5
    }
    subset = [rows_at_threshold[method] for method in ("Otsu", "
        Watershed", "StarDist_Python")]
    labels = [str(row["method"]) for row in subset]
    ap = [float(row["accuracy_mean"]) for row in subset]
    pq = [float(row["panoptic_quality_mean"]) for row in subset]
    x = np.arange(len(labels))
    figure, axis = plt.subplots(figsize=(7, 4.5))
    width = 0.36
    axis.bar(x - width / 2, ap, width, label="AP@0.50")
    axis.bar(x + width / 2, pq, width, label="PQ@0.50")
    axis.set_ylabel("Mean_score_on_official_test_split")
    axis.set_ylim(0, 1)
    axis.set_xticks(x, labels)
    axis.legend()
    axis.grid(axis="y", alpha=0.25)
    figure.tight_layout()
    figure.savefig(RESULTS / "figures" / "benchmark_ap_pq.png", dpi

```

```

        =200)
plt.close(figure)

def save_qualitative_panel(
    image_path: Path,
    min_distance: int,
    model: StarDist2D,
    tag: str,
) -> None:
    """Save raw/GT/predictions and TIFF labels for one selected test
        image."""

    image = imread(image_path)
    truth = imread(DATASET_ROOT / "test" / "masks" / image_path.name)
    otsu = segment_otsu(image, foreground="bright", min_size=20).
        labels
    watershed = segment_watershed(
        image, foreground="bright", min_size=20, min_distance=
            min_distance
    ).labels
    stardist, _, _, _ = predict_stardist(model, image)
    predictions = {
        "Otsu": otsu,
        "Watershed": watershed,
        "StarDist_Python": stardist,
    }
    label_dir = RESULTS / "selected_labels" / tag
    for method, labels in predictions.items():
        save_labels(label_dir / f"{method.lower().replace('_', '_')}.
            tif", labels)
    save_labels(label_dir / "ground_truth.tif", truth)
    figure, axes = plt.subplots(1, 5, figsize=(18, 4))
    panels = [("Input", image), ("Ground_truth", truth), *predictions
        .items()]
    for axis, (title, array) in zip(axes, panels, strict=True):
        axis.imshow(array, cmap="gray" if title == "Input" else "
            nipy_spectral")
        axis.set_title(title)
        axis.axis("off")
    figure.suptitle(f"Representative_{tag}_case_selected_by_StarDist_

```

```

        AP@0.50", y=1.02)
figure.tight_layout()
figure.savefig(RESULTS / "figures" / f"qualitative_{tag}.png",
               dpi=200, bbox_inches="tight")
plt.close(figure)

def main() -> None:
    ensure_dataset()
    RESULTS.mkdir(parents=True, exist_ok=True)
    (RESULTS / "figures").mkdir(exist_ok=True)
    all_train = paired_files("train")
    validation_indices = np.random.default_rng(42).choice(
        len(all_train), size=min(30, len(all_train)), replace=False
    )
    validation = [all_train[index] for index in sorted(
        validation_indices)]
    test = paired_files("test")
    selected_distance, validation_rows = choose_watershed_distance(
        validation)
    write_csv(RESULTS / "watershed_validation_sweep.csv",
              validation_rows)

    model = StarDist2D.from_pretrained("2D_versatile_fluo")
    # Build the TensorFlow graph before recording inference times.
    predict_stardist(model, imread(test[0][0]))
    rows: list[dict[str, object]] = []
    for image_path, mask_path in test:
        image = imread(image_path)
        truth = imread(mask_path)
        runs: list[tuple[str, np.ndarray, float, dict[str, object]]]
            = []
        start = time.perf_counter()
        otsu = segment_otsu(image, foreground="bright", min_size=20)
        runs.append(("Otsu", otsu.labels, time.perf_counter() - start
            , otsu.metadata))
        start = time.perf_counter()
        watershed = segment_watershed(
            image, foreground="bright", min_size=20, min_distance=
                selected_distance
        )

```

```

runs.append(
    ("Watershed", watershed.labels, time.perf_counter() -
     start, watershed.metadata)
)
stardist, elapsed, probability_threshold, nms_threshold =
    predict_stardist(model, image)
runs.append(
    (
        "StarDist_Python",
        stardist,
        elapsed,
        {
            "model": "2D_versatile_fluo",
            "pmin": 1.0,
            "pmax": 99.8,
            "probability_threshold": probability_threshold,
            "nms_threshold": nms_threshold,
        },
    )
)
for method, prediction, elapsed, metadata in runs:
    for metric in evaluate_at_thresholds(truth, prediction,
                                         IOU_THRESHOLDS):
        rows.append(
            {
                "image": image_path.name,
                "method": method,
                "runtime_s": elapsed,
                **metadata,
                **metric.to_dict(),
            }
        )
write_csv(RESULTS / "per_image_metrics.csv", rows)
summary = make_summary(rows)
write_csv(RESULTS / "summary_metrics.csv", summary)
make_figure(summary)
stardist_ap = sorted(
    (
        row
        for row in rows
        if row["method"] == "StarDist_Python" and float(row["

```

```

        threshold")) == 0.5
    ),
    key=lambda row: float(row["accuracy"]),
)
selected_positions = {"low": 0, "median": len(stardist_ap) // 2,
    "high": -1}
image_index = {path.name: path for path, _ in test}
selected_rows: list[dict[str, object]] = []
for tag, position in selected_positions.items():
    row = stardist_ap[position]
    image_path = image_index[str(row["image"])]
    save_qualitative_panel(image_path, selected_distance, model,
        tag)
    selected_rows.append(
        {
            "tag": tag,
            "image": image_path.name,
            "stardist_ap_iou_050": row["accuracy"],
            "selection_rule": (
                "min, median, or max StarDist AP@0.50 over the
                complete test split"
            ),
        }
    )
)
write_csv(RESULTS / "selected_cases.csv", selected_rows)
(RESULTS / "provenance.json").write_text(
    json.dumps(
        {
            "dataset": "Official StarDist DSB2018 archive",
            "archive_url": URL,
            "train_images_for_validation": len(validation),
            "test_images": len(test),
            "watershed_min_distance_selected_on_train":
                selected_distance,
            "stardist_model": "2D_versatile_fluo",
            "stardist_threshold_policy": "model_default
                thresholds; no test-set tuning",
            "device": "CPU",
            "qualitative_selection": "min/median/max StarDist
                AP@0.50 over all test images",
        }
    ),

```

```

        ensure_ascii=False,
        indent=2,
    )
    + "\n",
    encoding="utf-8",
)

if __name__ == "__main__":
    main()

```

C.4 Runner StarDist ImageJ/Fiji

Listing C.5: `FijiStarDistRunner.java`: gọi command plugin, xuất Label Image TIFF và log runtime.

```

/**
 * Headless, reproducible runner for the published ImageJ/Fiji
 * StarDist 2D command.
 *
 * <p>Usage: {FijiStarDistRunner input.tif label-output.tif [
 * nTiles]}. The
 * StarDist 0.3.0-scijava bundled fluorescent-nuclei model is used
 * with its
 * optimized thresholds: probability/score=0.479071 and overlap-NMS
 * =0.3.</p>
 */

import java.util.HashMap;
import java.util.Map;

import de.csbdresden.stardist.StarDist2D;
import net.imagej.Dataset;
import net.imagej.ImageJ;
import org.scijava.command.CommandModule;

public final class FijiStarDistRunner {
    private FijiStarDistRunner() {}

    public static void main(final String... args) throws Exception {
        if (args.length < 2 || args.length > 3) {

```

```

        throw new IllegalArgumentException(
            "Usage: FijiStarDistRunner input.tif label-output.tif
            [nTiles]"
        );
    }

    final int nTiles = args.length == 3 ? Integer.parseInt(args
        [2]) : 1;
    if (nTiles < 1) throw new IllegalArgumentException("nTiles
        must be >= 1");

    final ImageJ ij = new ImageJ();
    final Map<String, Object> parameters = new HashMap<>();
    parameters.put("input", (Dataset) ij.io().open(args[0]));
    parameters.put("modelChoice", "Versatile_(fluorescent_nuclei)");
    parameters.put("normalizeInput", true);
    parameters.put("percentileBottom", 1.0);
    parameters.put("percentileTop", 99.8);
    parameters.put("probThresh", 0.479071);
    parameters.put("nmsThresh", 0.3);
    parameters.put("outputType", "Label_Image");
    parameters.put("nTiles", nTiles);
    parameters.put("excludeBoundary", 2);
    parameters.put("roiPosition", "Automatic");
    parameters.put("verbose", false);
    parameters.put("showCsbdeepProgress", false);
    parameters.put("showProbAndDist", false);

    try {
        final long startNanos = System.nanoTime();
        final CommandModule module = ij.command().run(StarDist2D.
            class, false, parameters).get();
        final double elapsedSeconds = (System.nanoTime() -
            startNanos) / 1_000_000_000.0;
        final Dataset label = (Dataset) module.getOutput("label")
            ;
        if (label == null) throw new IllegalStateException("
            StarDist_produced_no_Label_Image_output.");
        ij.io().save(label, args[1]);
    }

```

```

        System.out.printf("INPUT=%s\nOUTPUT=%s\nMODEL=Versatile_(
        fluorescent_nuclei)%n" +
        "PROB_THRESH=0.479071\nNMS_THRESH=0.3\nNTILES=%d%
        nRUNTIME_S=%.6f\n",
        args[0], args[1], nTiles, elapsedSeconds);
    } finally {
        ij.context().dispose();
    }
    System.exit(0);
}
}

```